

Linear Regression

Aim

To implement a Linear Regression model in Python to predict the relationship between a dependent variable and one or more independent variables.

Procedure

1. Import the necessary libraries such as pandas, numpy, and sklearn.
2. Load the dataset into the program and examine it to understand the features and target variable.
3. Select the relevant independent variable(s) and the dependent variable for the regression.
4. Split the dataset into training and testing sets to evaluate model performance.
5. Create a Linear Regression model using sklearn.
6. Train the model on the training dataset to learn the relationship between features and target.
7. Use the trained model to predict the target variable for the test dataset.
8. Compare the predicted values with the actual values to assess accuracy.
9. Store or display the predicted values alongside actual values for interpretation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv('Salary.csv')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   YearsExperience  30 non-null    float64
 1   Salary          30 non-null    int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.head()
```

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

```
df.isnull().sum()
```

	0
YearsExperience	0
Salary	0

dtype: int64

```
df.describe()
```

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
```

```
features
```

```
array([[ 1.],
       [ 1.3],
       [ 1.5],
       [ 2. ],
       [ 2.2],
       [ 2.9],
       [ 3. ],
       [ 3.2],
       [ 3.2],
       [ 3.7],
       [ 3.9],
       [ 4. ],
       [ 4. ],
       [ 4.1],
```

```
[ 4.5],
[ 4.9],
[ 5.1],
[ 5.3],
[ 5.9],
[ 6. ],
[ 6.8],
[ 7.1],
[ 7.9],
[ 8.2],
[ 8.7],
[ 9. ],
[ 9.5],
[ 9.6],
[10.3],
[10.5]]
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=42)
```

```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```
model.score(x_train,y_train)
```

```
0.9645401573418146
```

```
model.score(x_test,y_test)
```

```
0.9024461774180497
```

```
model.coef_
```

```
array([[9423.81532303]])
```

```
model.intercept_
```

```
array([25321.58301178])
```

```
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
```

```
model=pickle.load(open('SalaryPred.model','rb'))
```

```
yr=int(input("Enter years of experience: "))
yr_of_exp_NP=np.array([[yr]])
Salary=model.predict(yr_of_exp_NP)
```

```
Enter years of experience: 50
```

```
print("Expected Salary for {} years of experience: {}".format(yr,Salary))
```

```
Expected Salary for 50 years of experience: [[496512.34916333]]
```

Result:

Hence, a Python program to perform Linear Regression has been implemented successfully, predicting the dependent variable from the independent variable(s) and generating predicted values for analysis.